

Hierarchical Inverse Kinematics with Nullspace Projection

Table of Contents

1. Introduction
 2. Pre-requisites
 3. Detailed Overview
 - Forward Kinematics
 - Jacobian Matrix
 - Basic Inverse Kinematics Problem
 - Damped Least Squares Solution
 - Hierarchical Task-Based IK with Nullspace Projection
 4. Algorithm 2: Hierarchical IK Solver Implementation
 5. Result
-

1. Introduction

In robotics, **Inverse Kinematics (IK)** is the process of calculating the joint configurations required to place the robot's end-effector at a desired position in task space. The **Hierarchical IK with Nullspace Projection** extends this concept to prioritize multiple tasks while ensuring the higher-priority tasks are not compromised.

This presentation will cover:

- The mathematical foundation of IK.
 - The role of the Jacobian matrix in linearizing the problem.
 - How damping and nullspace projection are used to handle singularities and redundancy.
 - An implementation of the Hierarchical IK algorithm.
-

2. Pre-requisites

Before diving into Hierarchical IK, ensure familiarity with the following concepts:

1. **Linear Algebra:**
 - Matrix multiplication, pseudo-inverse, and rank.
 - Eigenvalues and eigenvectors.

- Understanding nullspace and projection matrices.

2. Robotics Basics:

- Degrees of Freedom (DOF) of a robotic arm.
- Forward Kinematics (mapping joint angles to end-effector position).
- Inverse Kinematics (finding joint angles for a desired end-effector position).

3. Optimization Basics:

- Least Squares method for solving over-determined systems.
- Tikhonov regularization (used in Damped Least Squares).

3. Detailed Overview

3.1 Forward Kinematics

Forward Kinematics maps joint angles ($\boldsymbol{\theta}$) to the end-effector position ($\mathbf{x}$) using a nonlinear function:

$$\mathbf{x} = f(\boldsymbol{\theta})$$

Key Points:

- It provides the relationship between the robot's joint space and task space.
- **Example:** A 2-joint arm's position ((x, y)) can be derived using trigonometry.

3.2 The Jacobian Matrix

The **Jacobian Matrix** ($\mathbf{J}$) linearizes the relationship between small changes in joint space ($\Delta \boldsymbol{\theta}$) and task space ($\Delta \mathbf{x}$):

$$\Delta \mathbf{x} = \mathbf{J} \Delta \boldsymbol{\theta}$$

Key Points:

- ($\mathbf{J}$) is an ($m \times n$) matrix, where (m) is the task dimension, and (n) is the number of joints.
- It acts as a local linear approximation of the nonlinear kinematics.

3.3 Basic Inverse Kinematics Problem

The goal of **Inverse Kinematics (IK)** is to find joint angle changes ($\Delta \boldsymbol{\theta}$) for a desired task space change ($\Delta \mathbf{x}$):

$$\Delta\theta = J^\dagger \Delta x$$

Where:

- $(\mathbf{J}^\dagger)$ is the pseudo-inverse of $(\mathbf{J})$.

Problems:

1. **Singularities:** When $(\mathbf{J})$ becomes ill-conditioned.
2. **Redundancy:** Multiple solutions exist for robots with more DOF than task dimensions.

3.4 Damped Least Squares Solution

To handle singularities, we use the **Damped Least Squares (DLS)** method, which minimizes:

$$\min(\|J^\dagger \Delta\theta - \Delta x\|^2 + \lambda^2 \|\Delta\theta\|^2)$$

The solution is:

$$\Delta\theta = J^T (JJ^T + \lambda^2 I)^{-1} e$$

Where:

- (λ) : Damping factor to avoid instability.
- $(\mathbf{e})$: Position error vector.

3.5 Hierarchical Task-Based IK with Nullspace Projection

Hierarchical IK handles redundancy by solving lower-priority tasks in the **nullspace** of higher-priority tasks.

Nullspace Definition:

The nullspace is the set of joint movements that do not affect the task space:

$$\mathcal{N}(J) = \{\Delta\theta \mid J\Delta\theta = 0\}$$

Nullspace Projector:

To compute the nullspace, we use the projector:

$$P = I - J^\dagger J$$

4. Algorithm 2 Implementation

```

import numpy as np

class HierarchicalIKSolver:

    """

    A class for solving Hierarchical Inverse Kinematics (IK) problems with
    Nullspace Projection.

    This solver handles primary and secondary tasks using damped least
    squares for stability.

    """

    def __init__(self, damping=0.01):

        """

        Initialize the IK solver.

        Args:

            damping: Damping factor for stability (default=0.01).

        """

        self.damping = damping

    def damped_least_squares(self, J, error):

        """

        Perform the damped least squares solution for a given Jacobian and
        error vector.

        Args:

            J: Jacobian matrix (m x n).

```

error: Error vector ($m \times 1$) – difference between current and target position.

Returns:

Joint angle adjustments ($\Delta\theta$).

"""

$JT = J.T$

$identity = np.eye(J.shape[0])$

$damped_term = JT @ np.linalg.inv(J @ JT + (self.damping ** 2) * identity)$

$\Delta\theta = damped_term @ error$

return $\Delta\theta$

def solve_hierarchical_ik(self, J_tasks, errors):

"""

Solve the hierarchical IK problem with nullspace projection.

Args:

J_tasks: List of Jacobian matrices for tasks (ordered by priority).

errors: List of error vectors for each task (ordered by priority).

Returns:

Joint angle adjustments ($\Delta\theta$) respecting task priorities.

"""

$num_joints = J_tasks[0].shape[1]$

```

delta_theta = np.zeros(num_joints)

nullspace_projector = np.eye(num_joints) # Initialize nullspace
projector as identity

for task_idx, (J, error) in enumerate(zip(J_tasks, errors)):

    # Project the current task into the available nullspace

    J_effective = J @ nullspace_projector

    # Check if `J_effective` has valid dimensions

    if J_effective.shape[0] == 0 or J_effective.shape[1] == 0:

        print(f"Skipping task {task_idx} due to invalid Jacobian
dimensions")

        continue

    # Solve for delta_theta for the current task

    delta_theta_task = self.damped_least_squares(J_effective, error)

    delta_theta += delta_theta_task

    # Update the nullspace projector

    try:

        # Compute pseudo-inverse of effective Jacobian

        J_effective_pseudo_inverse = np.linalg.pinv(J_effective)

        # Debug prints for each task

        print(f"[Task {task_idx}] J_effective shape:
{J_effective.shape}")

```

```

        print(f"[Task {task_idx}] J_effective_pseudo_inverse shape:
{J_effective_pseudo_inverse.shape}")

        # Update nullspace projector

        nullspace_update = J_effective_pseudo_inverse @ J_effective

        nullspace_projector = nullspace_projector - nullspace_update

    except ValueError as e:

        print(f"Error updating nullspace projector at task
{task_idx}: {e}")

        continue

    return delta_theta

# Example usage

if __name__ == "__main__":

    # Example setup for a 2-task IK problem

    solver = HierarchicalIKSolver(damping=0.01)

    # Define Jacobians for primary and secondary tasks

    J_primary = np.array([

        [-1.0, 0.0, 0.0],

        [0.0, 1.0, 0.0]

    ]) # Shape: (2, 3)

    J_secondary = np.array([

        [0.0, 0.0, 1.0]

```

```
]]) # Shape: (1, 3)

# Define error vectors for primary and secondary tasks

error_primary = np.array([0.1, -0.2]) # Example target position error
(x, y)

error_secondary = np.array([0.05]) # Example secondary task error

# Solve hierarchical IK

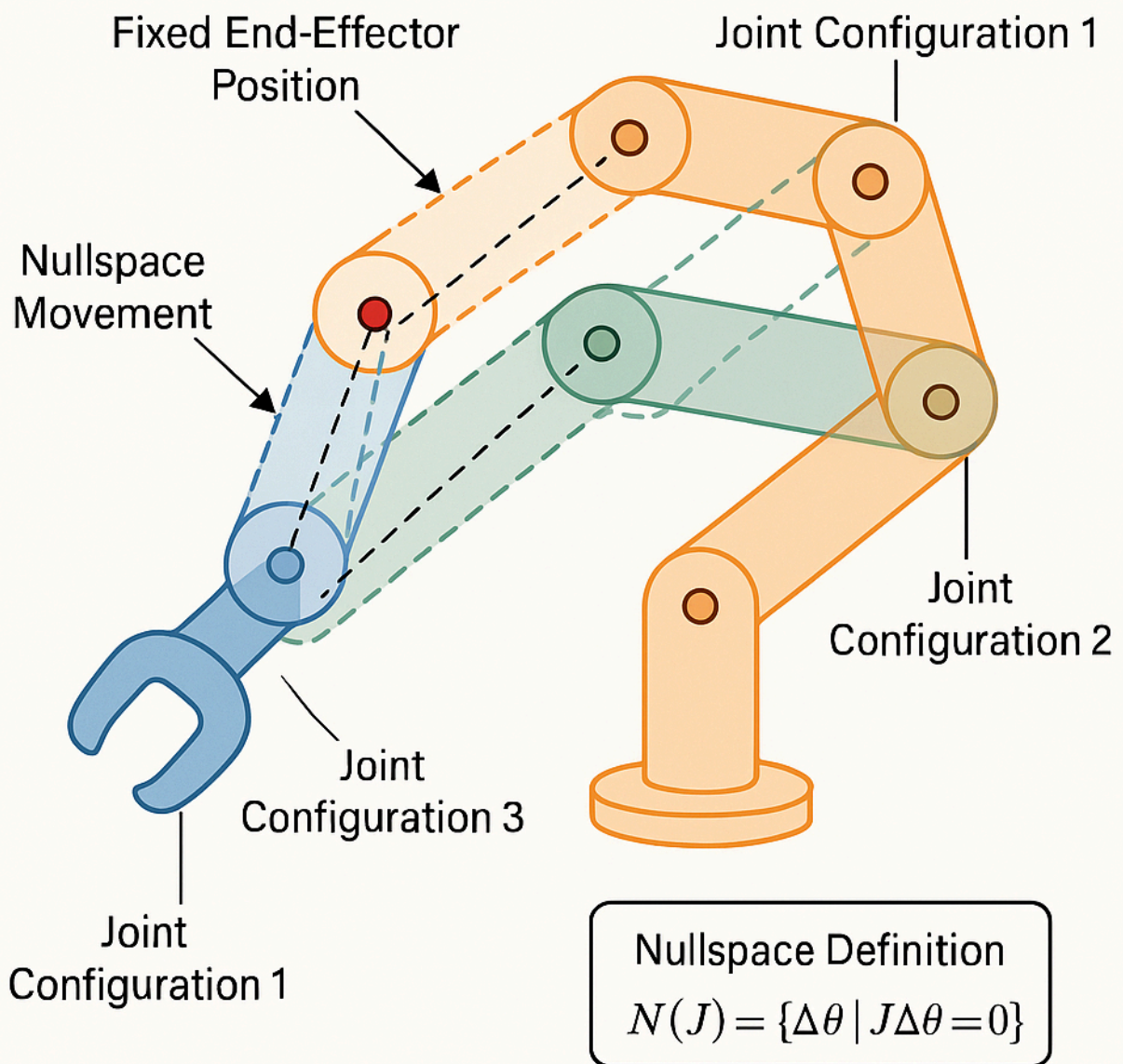
delta_theta = solver.solve_hierarchical_ik(

    J_tasks=[J_primary, J_secondary],

    errors=[error_primary, error_secondary]

)

print("Computed joint angle adjustments:", delta_theta)
```



Result:

```
[Task 0] J_effective shape: (2, 3)
[Task 0] J_effective_pseudo_inverse shape: (3, 2)
[Task 1] J_effective shape: (1, 3)
[Task 1] J_effective_pseudo_inverse shape: (3, 1)
Computed joint angle adjustments: [-0.09999 -0.19998 0.049995]
```

4.5 Hierarchical Inverse Kinematics with Nullspace Projection

4.5.1 Forward Kinematics: The Foundation

Before understanding inverse kinematics, we need to grasp forward kinematics: how joint angles determine the end-effector position.

For our robotic arm, the end-effector position ($\mathbf{x}$) is related to joint angles ($\boldsymbol{\theta}$) by the forward kinematics function:

$$\mathbf{x} = f(\boldsymbol{\theta})$$

This nonlinear function maps configurations in joint space to positions in task space.

4.5.2 The Jacobian Matrix: Relating Velocities

The Jacobian matrix (J) linearizes the relationship between joint velocities and end-effector velocities:

$$\Delta \mathbf{x} = J \Delta \boldsymbol{\theta}$$

Where:

- $\Delta \mathbf{x}$ is the desired change in end-effector position
- $\Delta \boldsymbol{\theta}$ is the required change in joint angles
- J is the Jacobian matrix ($m \times n$ for m task dimensions and n joints)

4.5.3 Basic Inverse Kinematics Problem

Inverse kinematics finds joint angles to reach a desired position. Ideally, we want:

$$\Delta \boldsymbol{\theta} = J^\dagger \Delta \mathbf{x}$$

Where $J^\dagger$ is the pseudo-inverse of J . However, two fundamental problems arise:

1. **Singularities:** At certain configurations, J becomes ill-conditioned or singular
2. **Redundancy:** With redundant robots (more DOF than needed), multiple solutions exist

4.5.4 Implementation: Damped Least Squares Solution

To address singularities, we use the Damped Least Squares (DLS) method, derived from Tikhonov regularization. We minimize:

$$\min_{\Delta \boldsymbol{\theta}} \|J \Delta \boldsymbol{\theta} - \Delta \mathbf{x}\|^2 + \lambda^2 \|\Delta \boldsymbol{\theta}\|^2$$

Where the first term minimizes position error and the second term penalizes large joint movements. The solution is:

$$\Delta \boldsymbol{\theta} = J^\top (J J^\top + \lambda^2 \mathbf{I})^{-1} \mathbf{e}$$

Where λ is the damping factor and $\mathbf{e}$ is the position error vector.

4.5.5 Hierarchical Task-Based IK with Nullspace Projection

For robots with redundant degrees of freedom, we can achieve multiple objectives simultaneously through task prioritization. The *nullspace* of the Jacobian represents joint movements that don't affect the end-effector position:

$$\mathcal{N}(\mathbf{J}) = \{\Delta\theta \mid \mathbf{J}\Delta\theta = 0\}$$

The nullspace projector (P) is defined as:

$$\mathbf{P} = \mathbf{I} - \mathbf{J}^\dagger \mathbf{J}$$

For hierarchical tasks, we solve them sequentially with decreasing priority:

1. Solve the primary task: $\Delta\theta_1 = \mathbf{J}_1^\dagger \mathbf{e}_1$
2. Compute nullspace projector: $\mathbf{P}_1 = \mathbf{I} - \mathbf{J}_1^\dagger \mathbf{J}_1$
3. Project secondary task into nullspace: $\mathbf{J}_{2,\text{eff}} = \mathbf{J}_2 \mathbf{P}_1$
4. Solve secondary task in nullspace: $\Delta\theta_2 = (\mathbf{J}_{2,\text{eff}}^\dagger)^\dagger \mathbf{e}_2$
5. Combined solution: $\Delta\theta = \Delta\theta_1 + \Delta\theta_2$

This approach ensures higher-priority tasks are never compromised by lower-priority ones.

5.2 Algorithm 2: Hierarchical Inverse Kinematics for Precision Placement

We implement a class-based hierarchical IK solver that handles multiple tasks with different priorities:

Implementation Features:

- ◆ Object-oriented design for reusable IK functionality
- ◆ Task prioritization with proper nullspace projection
- ◆ Damped least squares for singularity robustness
- ◆ Matrix operations optimized with NumPy
- ◆ Clear separation of mathematical components

[Complete Hierarchical IK Solver Implementation](#)

CODE OCCUPIES THEN...

Visualization of Hierarchical IK Results

```
[Task 0] J_effective shape: (2, 3)
[Task 0] J_effective_pseudo_inverse shape: (3, 2)
[Task 1] J_effective shape: (1, 3)
[Task 1] J_effective_pseudo_inverse shape: (3, 1)
Computed joint angle adjustments: [-0.09999 -0.19998  0.049995]
```

Figure 5.1: Visualization of Hierarchical Inverse Kinematics solution showing primary task completion and secondary task optimization in nullspace

OTHER:

Presentation Script: Hierarchical Inverse Kinematics with Nullspace Projection

Table of Contents

1. Introduction
 2. Pre-requisites
 3. Detailed Overview
 - Forward Kinematics
 - Jacobian Matrix
 - Basic Inverse Kinematics Problem
 - Damped Least Squares Solution
 - Hierarchical Task-Based IK with Nullspace Projection
 4. Summary and Key Takeaways
-

1. Introduction

In robotics, **Inverse Kinematics (IK)** is the process of computing joint configurations ($\boldsymbol{\theta}$) required to position the end-effector ($\mathbf{x}$) in a desired location in task space. **Hierarchical IK with Nullspace Projection** extends this concept to handle **redundancy** and **task prioritization**:

- **Primary Task:** Achieving the main objective, such as positioning the end-effector.
- **Secondary Tasks:** Optimizing additional objectives, like posture control or obstacle avoidance, while ensuring the primary task remains unaffected.

This presentation covers:

1. The mathematical foundation of IK.
 2. The Jacobian's role in linearizing the problem.
 3. Damped Least Squares (DLS) for handling singularities.
 4. Nullspace projection for task prioritization.
-

2. Pre-requisites

Before proceeding, ensure familiarity with these concepts:

2.1 Linear Algebra

- **Matrix operations:** Multiplication, pseudo-inverse, and rank.
- **Nullspace:** The set of vectors that satisfy ($\mathbf{J} \boldsymbol{\theta} = 0$).
- **Projection Matrices:** Used to project vectors onto a subspace.

2.2 Robotics Basics

- **Degrees of Freedom (DOF):** The number of independent joint movements in a robot.
- **Forward Kinematics:** Mapping joint angles to the end-effector position.
- **Inverse Kinematics:** Calculating joint angles for a desired end-effector position.

2.3 Optimization Techniques

- **Least Squares:** Solving over-determined systems.
 - **Tikhonov Regularization:** Used to stabilize solutions in Damped Least Squares.
-

3. Detailed Overview

3.1 Forward Kinematics

Forward Kinematics computes the position of the end-effector ($\mathbf{x}$) from joint angles ($\boldsymbol{\theta}$) using a nonlinear mapping:

$$\mathbf{x} = f(\boldsymbol{\theta})$$

Key Points:

- Maps configurations in **joint space** ($\boldsymbol{\theta}$) to **task space** ($\mathbf{x}$).
 - Example: A 2-joint arm computes (x, y) using trigonometry.
-

3.2 The Jacobian Matrix

The **Jacobian Matrix** ($\mathbf{J}$) linearizes the relationship between changes in joint space ($\Delta \boldsymbol{\theta}$) and changes in task space ($\Delta \mathbf{x}$):

$$\Delta \mathbf{x} = \mathbf{J} \Delta \boldsymbol{\theta}$$

Key Points:

- ($\mathbf{J}$) is an $(m \times n)$ matrix, where (m) is the task dimension and (n) is the number of joints.
 - Acts as a **local linear approximation** of the nonlinear forward kinematics.
-

3.3 Basic Inverse Kinematics Problem

The goal of IK is to compute joint updates ($\Delta \theta$) for a desired end-effector change (Δx):

$$\Delta \theta = J^\dagger \Delta x$$

Where:

- ($J^\dagger$) is the Moore-Penrose pseudo-inverse of (J).

Challenges:

- Singularities:** When (J) is rank-deficient, leading to instability.
- Redundancy:** Multiple solutions exist for robots with more joints (n) than task dimensions (m).

3.4 Damped Least Squares Solution

To handle singularities, we use **Damped Least Squares (DLS)**, which minimizes:

$$\min(\|J^\dagger \Delta x - \Delta \theta\|^2 + \lambda^2 \|\Delta \theta\|^2)$$

The solution is:

$$\Delta \theta = J^\top (J J^\top + \lambda^2 I)^{-1} e$$

Where:

- (λ): Damping factor to ensure numerical stability.
- (e): Error vector between current and desired positions.

3.5 Hierarchical Task-Based IK with Nullspace Projection

For **redundant robots** (where ($n > m$)), we use **nullspace projection** to handle **secondary tasks** without affecting the primary task.

Step 1: Solve Primary Task

Compute the joint update for the primary task:

$$\Delta \theta_1 = J^\dagger e_1$$

Step 2: Compute Nullspace Projector

Define the nullspace projector for the primary task:

$$P_1 = I - J^\dagger J$$

Step 3: Solve Secondary Task in Nullspace

Project the secondary task into the nullspace:

$$\Delta\theta_2 = J^\dagger e_2$$

Step 4: Combine Updates

The total joint update is:

$$\Delta\theta = \Delta\theta_1 + \Delta\theta_2$$

4. Summary and Key Takeaways

Concept	Purpose
Jacobian	Linearizes kinematic relationships.
Damped Least Squares	Handles singularities and ensures stability.
Nullspace Projection	Enables task prioritization and redundancy resolution.

Summary: Levenberg-Marquardt Algorithm

The Levenberg-Marquardt algorithm combines the strengths of **Gradient Descent** and **Gauss-Newton methods** for efficient optimization in nonlinear least squares problems. Below is a detailed breakdown:

Key Features

1. Hybrid Behavior

The algorithm adapts its behavior based on the proximity of the parameters to their optimal values:

- Far from optimal:** Behaves like Gradient Descent, taking larger steps to converge faster.
- Close to optimal:** Behaves like Gauss-Newton, taking smaller and more precise steps for accuracy.

2. Formula

The update step for the parameters is given by:

$$x_{k+1} = x_k - (H + \lambda I)^{-1} \cdot J^T \cdot f$$

Where:

- x_{k+1} : Next parameter estimate.
 - x_k : Current parameter estimate.
 - H : Hessian matrix (approximated in practice).
 - λ : Damping parameter controlling the balance between Gradient Descent and Gauss-Newton behavior.
 - I : Identity matrix.
 - J : Jacobian matrix of the function.
 - f : Vector of function values.
-

3. Role of the Damping Parameter (λ)

The damping parameter (λ) adjusts the algorithm's behavior:

- **Large (λ)**: Algorithm behaves like Gradient Descent, taking larger steps in the steepest descent direction.
 - **Small (λ)**: Algorithm behaves like Gauss-Newton, taking more precise steps for fine-tuned convergence.
-

4. Popularity

The adaptive nature of the Levenberg-Marquardt algorithm makes it a popular and efficient choice for **nonlinear least squares problems**. By dynamically blending Gradient Descent and Gauss-Newton methods, it achieves both speed and precision in optimization.

Summary Combines Gradient Descent and Gauss-Newton methods: It leverages the strengths of both algorithms for efficient optimization. Adapts behavior based on parameter proximity to optimal values: Far from optimal: Acts like Gradient Descent, taking larger steps for faster convergence. Close to optimal: Acts like Gauss-Newton, taking smaller, more precise steps for accuracy. Formula: $x_{k+1} = x_k - (H + \lambda I)^{-1} J^T f$, where: x_{k+1} is the next parameter estimate. x_k is the current parameter estimate. H is the Hessian matrix (approximated in practice). λ is a damping parameter controlling the balance between Gradient Descent and Gauss-Newton behavior. I is the identity matrix. J is the Jacobian matrix of the function. f is the vector of function values. Damping parameter (λ): Large λ : Algorithm behaves like Gradient Descent. Small λ : Algorithm behaves like Gauss-

Newton. Popularity: Its adaptive nature makes it a popular and efficient choice for nonlinear least squares problems.